

Intro to Fault Tolerance

CS230 Winter 2017

Basic Concepts

Dependability includes:

- **Availability:** Fraction of time system is “up”
- **Reliability:** System up long time (MTBF)
- **Safety:** Failures do not corrupt system/data
- **Maintainability:** How easy is system to fix?

Failure Models

Type of failure	Description
Crash failure	A server halts, but is working correctly until it halts
Omission failure <i>Receive omission</i> <i>Send omission</i>	A server fails to respond to incoming requests A server fails to receive incoming messages A server fails to send messages
Timing failure	A server's response lies outside the specified time interval
Response failure <i>Value failure</i> <i>State transition failure</i>	The server's response is incorrect The value of the response is wrong The server deviates from the correct flow of control
Arbitrary failure	A server may produce arbitrary responses at arbitrary times

Fail-stop versus Byzantine failures

Building Fault Tolerant Systems

Four major techniques:

1. Redundancy:

- Multiple copies of everything (processes & data)
- Quorums and voting (agreement)

2. Checkpointing

- Track state changes and be ready to "rollback"
- Consistent versus independent checkpoints

3. Message logging (and checkpointing)

- Augment checkpoints with dynamic event logs

4. Transactions

- Rewrite apps to be fault tolerant via abort-retry

Other Issues

Designers of FT systems also need to handle:

- Detecting faults
 - Heartbeats, voting, CRCs, ...
- Recovering lost data and/or computation
 - Checkpoints, other replicas, ...
- Coping with interactions with the “outside world”
 - Outside world: Components outside your control
 - Problem: Cannot just roll them back (no “Oops!”)

Replication (Active vs Passive)

Idea: Replicate key data and/or resources

- If data/resources are lost, use remaining copies

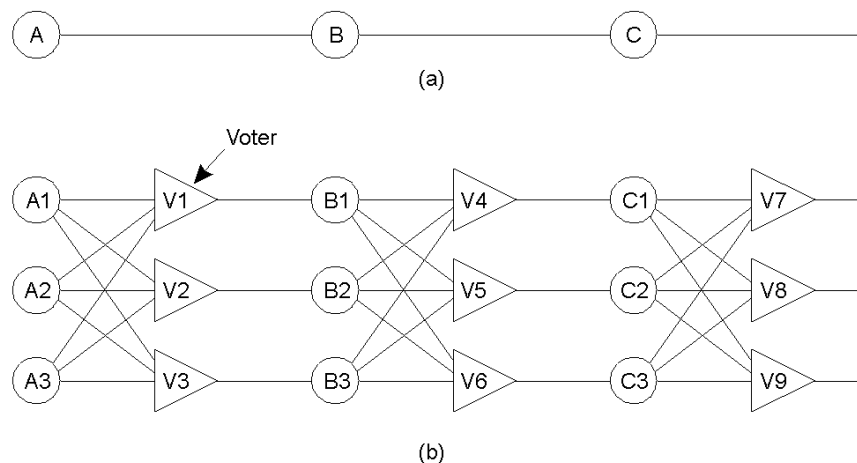
Active replication

- All processes/data are replicated (e.g., TMR)
- All replicas perform same set of ops (multicast)
- Need an agreement algorithm (e.g., voting)

Passive replication

- Master-slave replication
- Only master does work normally → pushes state

Failure Masking by Redundancy



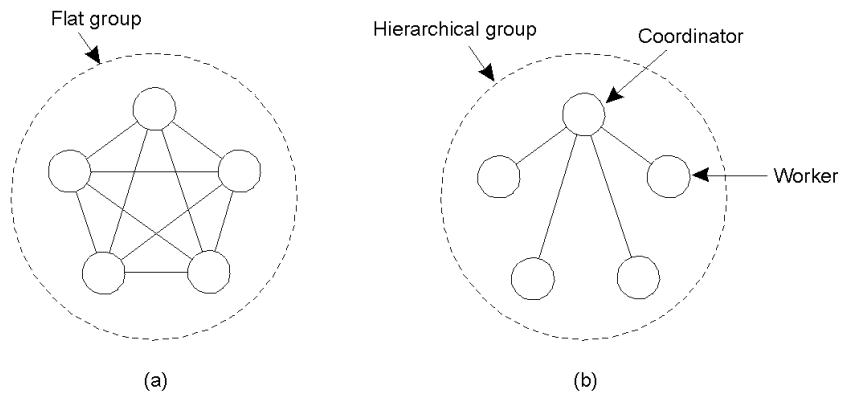
Triple modular redundancy.

Implementing Active Replication

Basic idea:

- Create N copies of each service/resource
- Ensure all ops done in same order everywhere
 - One idea $\rightarrow$ reliable ordered multicast (hard, slow)
- If a node fails, re-replicate from surviving node
- Questions:
 - In what ways does this model restrict generality?
 - What if replicas do not agree on outcome of op?
 - How do we handle Byzantine failures?
 - How do we distinguish server failure from dropped message?

Flat Groups versus Hierarchical Groups



- a) Communication in a flat group.
 b) Communication in a simple hierarchical group

Voting and Quorums

Issue: Replicas might not agree on outcome

- Non-determinism, faults, lost messages, ...
- Need to agree on ultimate outcome

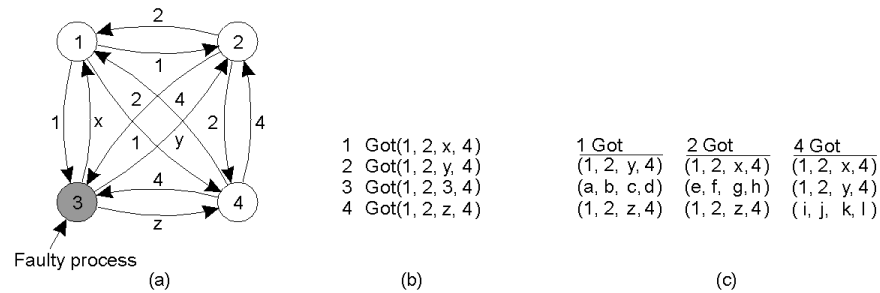
Typical solution: Voting

- Define “read” and “write” quorums
 - Typical: $R=1$, $W=N$ or $R = W = (N+1)/2$
- N replicas compare their outcomes
- Pick outcome of operation based on quorum vote

Many variants of voting schemes

- Check literature ...

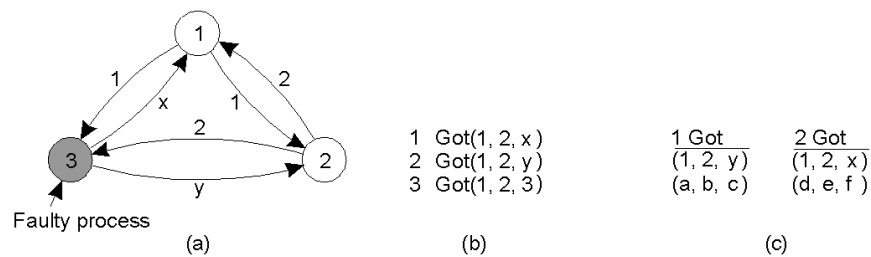
Byzantine Agreement (review)



The Byzantine generals problem for 3 loyal generals and 1 traitor.

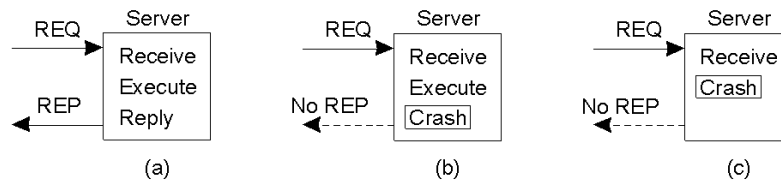
- a) The generals announce their troop strengths (in units of 1 kilosoldiers).
- b) The vectors that each general assembles based on (a)
- c) The vectors that each general receives in step 3.

Byzantine Agreement



The same as in previous slide, except now with 2 loyal generals and one traitor.

Lost Requests vs Server Crashes



A server in client-server communication

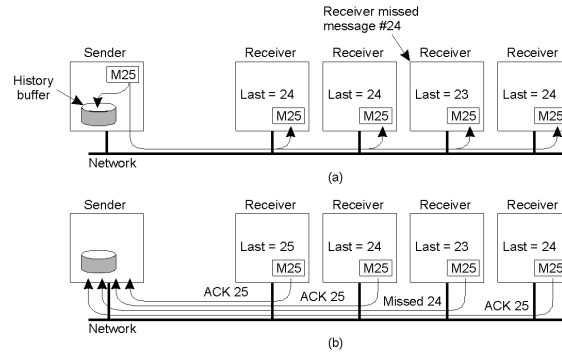
- a) Normal case
- b) Crash after execution
- c) Crash before execution

Server Crash Strategies

Client	Server						
	Strategy M → P			Strategy P → M			
	Reissue strategy	MPC	MC(P)	C(MP)	PMC	PC(M)	C(PM)
	Always	DUP	OK	OK	DUP	DUP	OK
	Never	OK	ZERO	ZERO	OK	OK	ZERO
	Only when ACKed	DUP	OK	ZERO	DUP	OK	ZERO
	Only when not ACKed	OK	ZERO	OK	OK	DUP	OK

Different combinations of client and server strategies in the presence of server crashes.

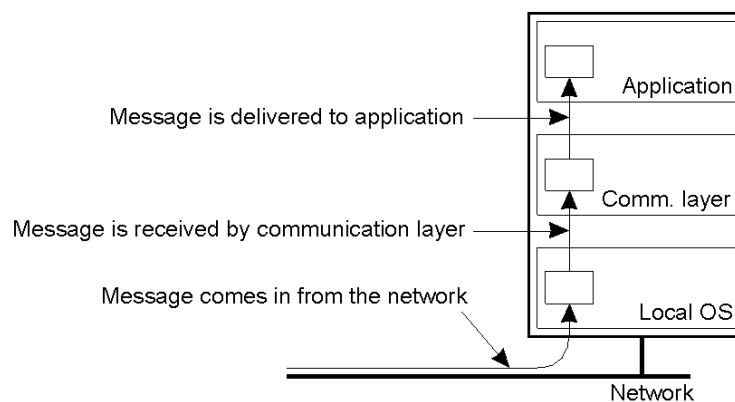
Basic Reliable-Multicasting Schemes



A simple solution to reliable multicasting when all receivers are known and are assumed not to fail

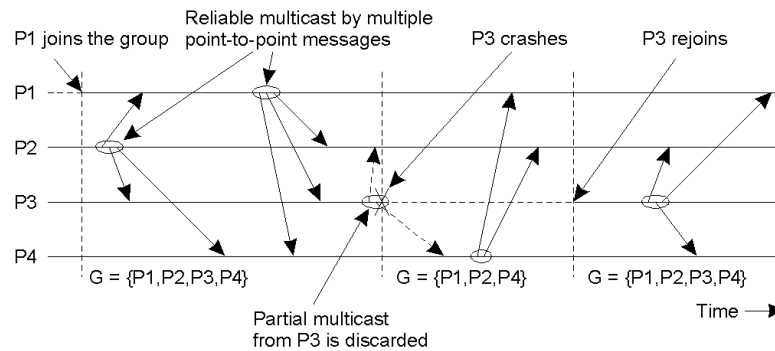
- a) Message transmission
- b) Reporting feedback

Virtual Synchrony (Ordered Multicast)



The logical organization of a distributed system to distinguish between message receipt and message delivery

Virtual Synchrony



The principle of virtual synchronous multicast.

Implementing Virtual Synchrony

Multicast	Basic Message Ordering	Total-ordered Delivery?
Reliable multicast	None	No
FIFO multicast	FIFO-ordered delivery	No
Causal multicast	Causal-ordered delivery	No
Atomic multicast	None	Yes
FIFO atomic multicast	FIFO-ordered delivery	Yes
Causal atomic multicast	Causal-ordered delivery	Yes

Six different versions of virtually synchronous reliable multicasting.

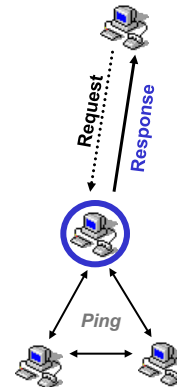
Passive Replication

Master-slave replication

- Only “master” computes
- Slaves receive updates
- If master dies, pick new leader

Fail-over issues:

- New master might take a while to bring up to date (apply updates)
 - Option: Checkpoint/restore
 - Option: Checkpoint/apply logged ops
- Clients must switch to new master
 - Preferably seamless, invisible, fast
- Failover usually slower than active replication



Distributed Checkpointing

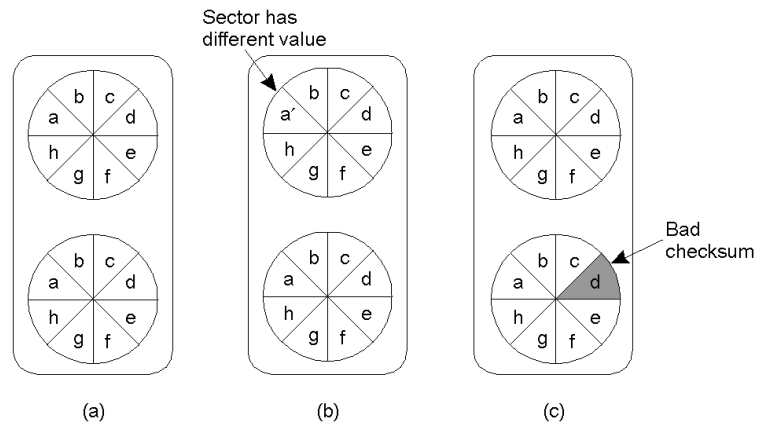
Idea:

- Periodically record state of process(es)
- If process fails, can restart from checkpoint.

Issues :

- How often should you take checkpoints?
- Where do we store checkpoints?
- What state must be saved in a checkpoint?
- Do we coordinate the N checkpoints?
 - What can happen if we do not coordinate?
- How much overhead will checkpointing add?

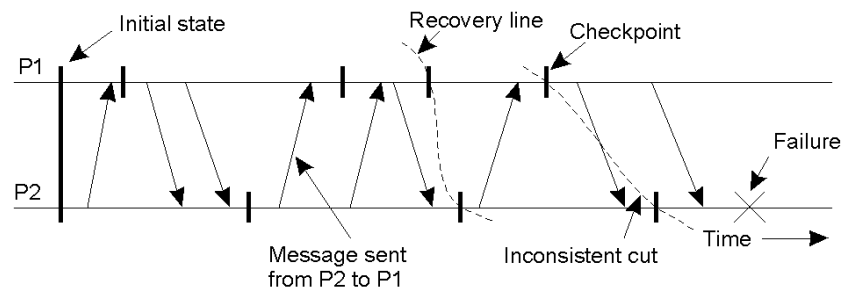
Stable Storage (e.g., RAID)



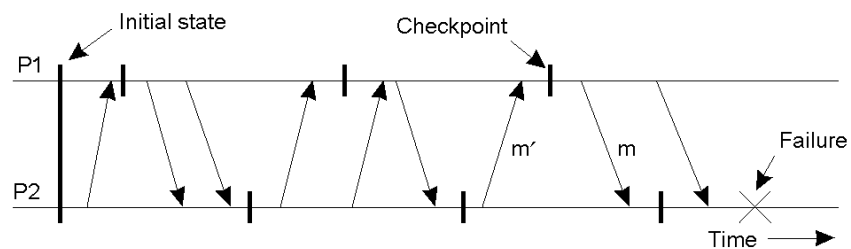
- a) Stable Storage
- b) Crash after drive 1 is updated
- c) Bad spot

Checkpointing

A recovery line.



Independent Checkpointing



Issue: Domino effect.

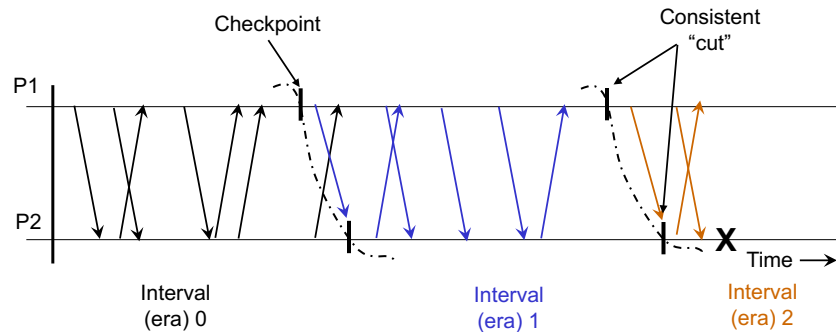
Consistent Checkpointing

Problem: Independent checkpoints can lead to cascading rollbacks (domino effect)

Idea: Eliminate domino effect by only taking checkpoint that are “globally consistent”

- Coordinator takes checkpoint and updates “era”
- Coordinator tags outgoing messages w/ new “era”
- If you receive a message from “newer era” → take a checkpoint
- Can send NULL messages to force checkpoints

Consistent Checkpointing



Discussion

What state do we need to save?

- All state a process depends on:
 - Address space, OS state, timer state, machine state, message layer state, ...
 - Virtual machine technology can simplify problem

How much overhead does checkpointing add?

- If done well, only ~10% failure-free overhead (app-dependent, interval-dependent)

How can we keep checkpointing overhead low?

- Incremental checkpoints (copy-on-write)
- Increase checkpoint interval (tradeoff)

Discussion (cont'd)

Coordinated vs independent checkpoints

- Extra overhead of coordination is small (surprise!)
- Major hit is the synchronized hit on fileserver

What about outside world?

- Cannot rollback outside world
- Option: Checkpoint state before/after interaction

What are some potential problems:

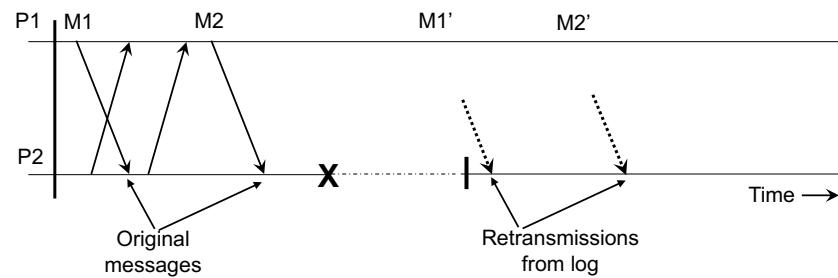
- Failure forces rollback of ALL processes
- Hard to checkpoint all important information
- Interactions with outside world are ugly

Message Logging and Checkpointing

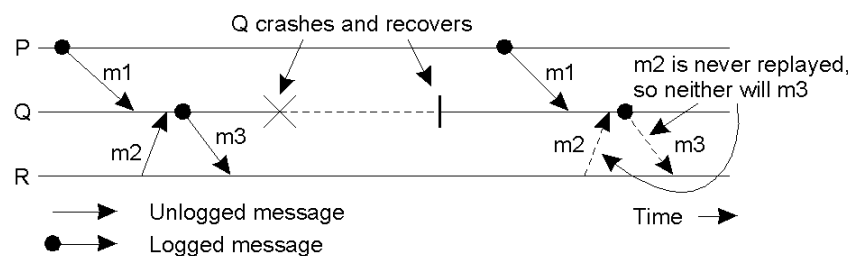
Idea: Enhance checkpointing w/ logs

- Log all events (e.g., messages!)
- Upon failure
 - Rollback only failed process
 - Replay events to process in same order as originally
 - Cull retransmissions sent during original execution
- Assumptions:
 - Process execution is deterministic
 - All important events can be logged
 - Fail-stop processing

Message Logging Implementation



Message Logging: Orphans



Incorrect replay of messages after recovery,
leading to an orphan process.

Forms of Message Logging

Pessimistic Logging:

- Log events before applying them to process
- Problem: Slows failure-free performance
- Benefit : Can always recover

Optimistic Logging:

- Log events in parallel with handing to process
- Faster but leaves small window of vulnerability

Sender-based versus receiver-based logging

Logging in volatile storage

Discussion

What events can/should we log?

- What problems arise?

Does this help interactions w/ outside world?

- Why or why not?

What performance overheads can we expect?

- Better or worse than checkpointing alone?

Transparent versus programmer-induced?

Transaction-Based Fault Tolerance

Idea: Expose fault tolerance issues to apps

- App writers write in terms of “transactions”
- If a fault occurs – abort transaction(s) and retry

Issue: Getting N replicas to commit/abort

Solution: Two-phase commit

- One replica designated “coordinator”
- Require same semantics as database transactions (ACID)
- Tricky part: Distributed agreement on commit/abort

ACID properties of transactions

In the context of transaction processing, the acronym *ACID* refers to the four key properties of a transaction:

atomicity,
consistency,
isolation, and
durability.

ACID properties of transactions

- **Atomicity:** All changes to data are performed as if they are a single operation. That is, all the changes are performed, or none of them are. For example, in an application that transfers funds from one account to another, the atomicity property ensures that, if a debit is made successfully from one account, the corresponding credit is made to the other account.
- **Consistency:** Data is in a consistent state when a transaction starts and when it ends. For example, in an application that transfers funds from one account to another, the consistency property ensures that the total value of funds in both the accounts is the same at the start and end of each transaction.

ACID properties of transactions

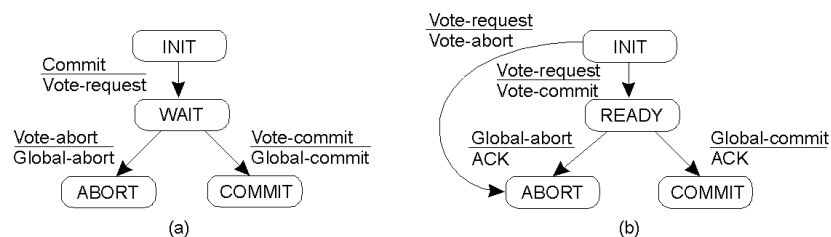
- **Isolation:** The intermediate state of a transaction is invisible to other transactions. As a result, transactions that run concurrently appear to be serialized. For example, in an application that transfers funds from one account to another, the isolation property ensures that another transaction sees the transferred funds in one account or the other, but not in both, nor in neither.
- **Durability:** After a transaction successfully completes, changes to data persist and are not undone, even in the event of a system failure. For example, in an application that transfers funds from one account to another, the durability property ensures that the changes made to each account will not be reversed.

Two-Phase Commit

Two-phase commit algorithm:

1. Coordinator sends "PREPARE TO COMMIT"
2. Cohorts write current state and "TENT_COMMIT" record in log
3. Cohorts reply to coordinator with "READY TO COMMIT"
4. If all are ready, coordinator writes "COMMIT" record to log, else ABORT
5. If transaction commits, coordinator sends out "COMMITTED" message

Two-Phase Commit



- a) The finite state machine for the coordinator in 2PC.
 b) The finite state machine for a participant.

Two-Phase Commit

State of Q	Action by P
COMMIT	Make transition to COMMIT
ABORT	Make transition to ABORT
INIT	Make transition to ABORT
READY	Contact another participant

Actions taken by a participant *P* when residing in state *READY* and having contacted another participant *Q*.

Summary

Fault tolerance is HARD:

– Many tradeoffs:

- Resources dedicated to FT versus performance
- User transparency versus performance
- Failure-free overhead versus recovery time
- Availability versus “correctness” (network partitions)

Replication versus logging versus transactions

Byzantine fault tolerance particularly hard